

Geeknito LMS Backend API Documentation

Welcome to the professional API documentation for the Geeknito LMS (Learning Management System) backend. This document provides a complete, developer-friendly guide to all implemented endpoints, request/response models (DTOs), authentication rules, authorization roles, query parameters, error definitions, and seeder test data.

Table of Contents

Table of Contents	1
System Architecture & Security	3
Authorized Roles.....	3
Global Headers & Error Handling.....	3
Request Headers.....	3
Standard Error Responses	3
Summary Table & Endpoint Metrics	4
API Endpoints by Module	5
Authentication.....	5
1. Login User.....	5
2. Register New Account	5
Users.....	6
1. Retrieve User Directory.....	6
Categories	7
1. Create Category (Public/Learner Namespace)	7
2. Get All Active Categories.....	8
3. Update Category (Admin Namespace).....	8
4. Delete Category (Admin Soft Delete).....	8
Courses	8
1. Public Course Catalog Page	8
2. Get Public Course Detail by Slug.....	9
3. Create Course (Admin Dashboard)	10
4. Delete Course (Soft or Hard Delete)	10
Curriculum (Modules & Submodules)	11
1. Public / Learner-Facing Curriculum Get	11
2. Create Module under Course	11
3. Create Submodule under Module	12
Content Blocks.....	12
1. Public Content Block Fetch for Submodule	12

2. Create Content Block (Admin Authoring).....	12
Files.....	13
1. Secure Binary File Upload.....	13
2. Download File	13
Enrollments	14
1. Enroll Currently Authenticated Student in Course	14
2. Get My Active Course Enrollments	14
3. Enroll in Batch (Direct Cohort Invite URL mapping).....	15
Batches.....	15
1. Create Student Cohort (Batch)	15
2. Get My Cohorts (Contextual Feed)	16
3. Assign Student to Batch	16
4. Remove Student from Batch	16
Assignments	17
1. Create Assignment.....	17
2. Get My Assignments (Student Feed)	18
3. Bind Batches to Assignment (Cohort Targeting).....	18
Questions	18
1. Create Question.....	18
2. Get Questions by Assignment	20
Submissions & Auto-Grading	20
1. Initialize (Start) Exam Session.....	20
2. Submit Active Session (MCQ Auto-Grading Core)	21
3. Save Draft (Draft Persistence)	21
4. Post manual grades (Teacher Evaluation)	22
Certificates	22
Export.....	23
1. Export Assignment Results to Excel Spreadsheet.....	23
2. Export Cohort (Batch) Student Directory to Excel	23
Quality Assurance Checklist.....	24

System Architecture & Security

The Geeknito LMS is a secure, role-based, enterprise-grade REST API backend built using Spring Boot. Security is enforced through custom JWT authentication filter middleware.

Authorized Roles

The system distinguishes between three primary user roles:

- Admin (ADMIN): Full administrative control over all courses, categories, system files, global users, cohorts, and grading records.
- Teacher (TEACHER): Authoring rights for courses, curriculum modules, submodule content, assignments, and questions. Full cohort (batch) management and manual submission grading privileges.
- Student (STUDENT): Access to public/learner course catalogs, public active categories, course self-enrollment, interactive assignments, MCQ/essay submissions, and custom performance metrics.

Global Headers & Error Handling

Request Headers

All secured endpoints require the standard Authorization bearer header:

```
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json
```

Standard Error Responses

Errors follow a consistent structure as represented by the backend's standard exception handler:

```
{
  "timestamp": "2026-07-13T18:47:40Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed for field 'title'",
  "path": "/api/v1/assignments"
}
```

HTTP Status	Name	Cause / Trigger
400	Bad Request	Missing required parameters, field validation failure, or negative numeric bounds.
401	Unauthorized	Missing JWT bearer token, invalid token signature, or expired user session.
403	Forbidden	The authenticated user role (e.g. STUDENT) lacks privileges for admin/teacher routes.
404	Not Found	Reference record (User ID, Course ID, Batch ID, Submission ID) does not exist in the database.

HTTP Status	Name	Cause / Trigger
409	Conflict	Unique database index constraint violation (e.g. registering an already existing email address).
500	Internal Error	Disk file write failures, DB transaction rollbacks, or unexpected server exceptions.

Summary Table & Endpoint Metrics

Below is a breakdown of all 76 endpoints mapped across the 16 active RestControllers in the Geeknito LMS codebase.

Module	Endpoint Count	Public APIs	JWT APIs	Admin APIs	Teacher APIs	Student APIs
Authentication	2	2	0	0	0	0
Users	1	0	1	1	1	1
Categories	10	1	9	10	10	1
Courses	10	5	5	5	5	5
Curriculum	3	0	3	3	3	0
Content Blocks	4	0	4	4	4	0
Files	2	1	1	1	1	1
Enrollments	5	0	5	1	1	4
Batches	11	0	11	9	9	2
Assignments	12	0	12	12	12	3
Questions	5	0	5	5	5	1
Submissions	9	0	9	9	9	6
Export	2	0	2	2	2	0
Total	76	9	67	61	61	24

Note: Roles specified denote maximum availability. Student APIs can access student-specific segments, while Admins/Teachers enjoy global overrides.

API Endpoints by Module

Authentication

Handles user registration and secure credential validation with on-the-fly BCrypt migration for legacy records.

Note: Geeknito LMS uses stateless JWT authentication. Logout is performed entirely on the client by removing the stored JWT token. No backend logout endpoint exists.

1. Login User

Endpoint: POST /auth/login

Purpose: Validates user credentials and issues a cryptographically secure JWT token containing the user's ID, email, and resolved role.

Authentication: Public (No JWT required)

Authorized Roles: Any visitor

Request Headers:

- Content-Type: application/json

Request Body (LoginRequest):

```
{
  "email": "student@educorp.com",
  "password": "studentpass123"
}
```

Successful Response:

- Status Code: 200 OK
- JSON Example (AuthResponse):

```
{
  "id": 2,
  "fullName": "Emma Watson",
  "email": "student@educorp.com",
  "role": "student",
  "token":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MiwiIjoic3R1ZGVudEB1ZHVjb3JwLmNvbSIsInJvbGU0iJzdHVkZW50In0.signature"
}
```

Error Responses:

- 400 Bad Request: Email address is required.
- 401 Unauthorized: Invalid credentials. User does not exist / Invalid email or password.
- 403 Forbidden: This user account has been deactivated.

Seeder Test Data:

- Student: student@educorp.com / studentpass123
- Teacher: teacher@educorp.com / teacherpass123
- Admin: admin@educorp.com / adminpass123

2. Register New Account

Endpoint: POST /auth/register

Purpose: Creates a new active user account. Resolves roles securely based on email format or provided role parameters.

Authentication: Public (No JWT required)

Authorized Roles: Any visitor

Request Headers:

- Content-Type: application/json

Request Body (RegisterRequest):

```
{
  "fullName": "Academy Learner",
  "email": "learner@educorp.com",
  "password": "securepassword123",
  "role": "student"
}
```

Successful Response:

- Status Code: 201 Created
- JSON Example (AuthResponse):

```
{
  "id": 31,
  "fullName": "Academy Learner",
  "email": "learner@educorp.com",
  "role": "student",
  "token":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MzEsInN1YiI6Imx1YXJuZXJAZWR1Y29ycC5jb20iLCJyb2x1Ijoic3R1ZGVudCJ9.signature"
}
```

Error Responses:

- 400 Bad Request: Email address is required / Password must be at least 6 characters.
- 409 Conflict: A user with this email address already exists.

Notes: If the email address contains the substring "admin", the role resolves as admin automatically to facilitate enterprise bootstrapping.

Users

Provides active directories and search utilities for managing the academy's user base.

1. Retrieve User Directory

Endpoint: GET /users

Purpose: Returns a list of users registered in the system, optionally filtered by user role.

Authentication: JWT Required

Authorized Roles: Admin, Teacher, Student

Request Headers:

- Authorization: Bearer <JWT_TOKEN>

Query Parameters:

- role (String, Optional): Filter directory by role (admin, teacher, student).

Successful Response:

- Status Code: 200 OK
- JSON Example (List<UserResponseDTO>):

```
[
  {
    "id": 2,
    "fullName": "Emma Watson",
    "email": "student@educorp.com",
    "role": "student",
    "isActive": true,
    "createdAt": "2026-07-13T18:47:40",
    "updatedAt": "2026-07-13T18:47:40"
  }
]
```

Categories

Manages course taxonomic categories. Dual controllers map learner public listings vs. admin configurations.

1. Create Category (Public/Learner Namespace)

Endpoint: POST /api/categories

Purpose: Creates a new course category with color and visual icon.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Request Body (CategoryRequestDTO):

```
{
  "name": "Cloud Architecture",
  "icon": "☁",
  "description": "Master public and private cloud design.",
  "color": "#0284C7",
  "isActive": true
}
```

Successful Response:

- Status Code: 201 Created
- JSON Example (CategoryResponseDTO):

```
{
  "id": 1,
  "name": "Cloud Architecture",
  "icon": "☁",
  "description": "Master public and private cloud design.",
  "color": "#0284C7",
  "isActive": true,
  "createdAt": "2026-07-13T18:47:40",
  "updatedAt": "2026-07-13T18:47:40",
}
```

```
"courseCount": 0
}
```

2. Get All Active Categories

Endpoint: GET /api/categories

Purpose: Public endpoint displaying all active categories for the learner catalog.

Authentication: Public

Successful Response:

- Status Code: 200 OK

3. Update Category (Admin Namespace)

Endpoint: PUT /categories/{id}

Purpose: Modifies structural details of a category by ID.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Path Variables:

- id (Long, Required): Category database primary key.

Request Body: Same as CategoryRequestDTO.

Successful Response:

- Status Code: 200 OK

4. Delete Category (Admin Soft Delete)

Endpoint: DELETE /categories/{id}

Purpose: Soft-deletes a category by setting its active state to false, preserving database rows.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Successful Response:

- Status Code: 204 No Content

Courses

The core course catalog framework. Maps learner-facing catalog lists to admin creation utilities.

1. Public Course Catalog Page

Endpoint: GET /catalog/courses or GET /courses

Purpose: Returns a paginated list of course cards suitable for public viewing on the landing page.

Authentication: Public

Query Parameters:

- page (int, default: 0): Page index.
- size (int, default: 10): Items per page.
- sortBy (String, default: "id"): Sort field.

- direction (String, default: "ASC"): Sort direction (ASC/DESC).

Successful Response:

- Status Code: 200 OK
- JSON Example (Page<CourseCatalogCardDTO>):

```
{
  "content": [
    {
      "id": 1,
      "title": "AWS Certified Solutions Architect",
      "slug": "aws-solutions-architect",
      "thumbnailUrl": "https://images.unsplash.com/photo-1451187580459-43490279c0fa?q=80&w=600&auto=format&fit=crop",
      "shortDescription": "Design highly available, cost-efficient systems.",
      "level": "Intermediate",
      "duration": "40 hours",
      "categoryName": "Cloud Architecture",
      "isPublished": true,
      "moduleCount": 2,
      "submoduleCount": 4
    }
  ],
  "totalPages": 1,
  "totalElements": 1,
  "size": 10,
  "number": 0
}
```

2. Get Public Course Detail by Slug

Endpoint: GET /catalog/courses/{slug} or GET /courses/{slug}

Purpose: Resolves detailed course descriptors (used to construct Course Landing pages).

Authentication: Public

Path Variables:

- slug (String, Required): URL-safe descriptive identifier.

Successful Response:

- Status Code: 200 OK
- JSON Example (CourseCatalogDetailDTO):

```
{
  "id": 1,
  "title": "AWS Certified Solutions Architect",
  "slug": "aws-solutions-architect",
  "thumbnailUrl": "https://images.unsplash.com/photo-1451187580459-43490279c0fa?q=80&w=600&auto=format&fit=crop",
  "shortDescription": "Design highly available, cost-efficient systems.",
  "description": "This comprehensive, production-grade course is designed for solutions architects...",
  "level": "Intermediate",
  "duration": "40 hours",
  "categoryName": "Cloud Architecture",
  "isPublished": true,
  "modules": []
}
```

```
}
```

3. Create Course (Admin Dashboard)

Endpoint: POST /admin/courses

Purpose: Registers a new course within a category.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Request Body (CourseRequestDTO):

```
{
  "title": "Modern Backend Architectures",
  "slug": "modern-backend-architectures",
  "shortDescription": "Master microservices and databases.",
  "description": "Learn event-driven architectures and API gateways.",
  "level": "Advanced",
  "duration": "30 hours",
  "thumbnailUrl": "https://images.unsplash.com/photo-1451187580459-43490279c0fa",
  "categoryId": 1,
  "isActive": true,
  "isPublished": false
}
```

Successful Response:

- Status Code: 201 Created
- JSON Example (CourseResponseDTO):

```
{
  "id": 21,
  "title": "Modern Backend Architectures",
  "slug": "modern-backend-architectures",
  "shortDescription": "Master microservices and databases.",
  "description": "Learn event-driven architectures and API gateways.",
  "level": "Advanced",
  "duration": "30 hours",
  "thumbnailUrl": "https://images.unsplash.com/photo-1451187580459-43490279c0fa",
  "categoryId": 1,
  "categoryName": "Cloud Architecture",
  "isActive": true,
  "isPublished": false,
  "createdAt": "2026-07-13T18:47:40",
  "updatedAt": "2026-07-13T18:47:40"
}
```

4. Delete Course (Soft or Hard Delete)

Endpoint: DELETE /admin/courses/{id}

Purpose: Removes a course. Supports soft-delete by default, or absolute database deletion.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Query Parameters:

- hard (boolean, default: false): Set to true to purge completely from database.

Successful Response:

- Status Code: 244 No Content

Curriculum (Modules & Submodules)

Controls course content structure (Modules act as folders, Submodules represent specific lessons).

1. Public / Learner-Facing Curriculum Get

Endpoint: GET /courses/{courseId}/curriculum or GET /catalog/courses/{courseId}/curriculum

Purpose: Fetches the modular curriculum structure (modules containing nested submodules) of a course for learner interfaces.

Authentication: Public

Path Variables:

- courseId (Long, Required): Database course record ID.

Successful Response:

- Status Code: 200 OK
- JSON Example (List<ModuleResponseDTO>):

```
[
  {
    "id": 1,
    "title": "Foundations of Cloud Storage",
    "description": "Explore high availability systems.",
    "moduleOrder": 1,
    "submodules": [
      {
        "id": 1,
        "title": "Intro to S3 Buckets",
        "description": "Set up secure buckets.",
        "submoduleOrder": 1
      }
    ]
  }
]
```

2. Create Module under Course

Endpoint: POST /admin/courses/{courseId}/modules

Purpose: Registers a curriculum parent folder.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Request Body (ModuleRequestDTO):

```
{
  "title": "Core Architectural Design Patterns",
  "description": "Master CQRS and event-driven patterns.",
  "moduleOrder": 1,
  "isActive": true
}
```

```
}
```

Successful Response:

- Status Code: 201 Created

3. Create Submodule under Module

Endpoint: POST /admin/courses/{courseId}/modules/{moduleId}/submodules

Purpose: Adds a lesson node under a module.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Request Body (SubmoduleRequestDTO):

```
{
  "title": "Designing Event Sourcing Pipelines",
  "description": "Understand immutable state events.",
  "submoduleOrder": 1,
  "isActive": true
}
```

Successful Response:

- Status Code: 201 Created

Content Blocks

Enables authors to map educational content types (Video, Rich Text Text, Quiz, Embedded Link, File Upload instructions) directly to lesson submodules.

1. Public Content Block Fetch for Submodule

Endpoint: GET /courses/submodules/{submoduleId}/contents or GET /catalog/courses/submodules/{submoduleId}/contents

Purpose: Serves active content assets to students navigating lessons.

Authentication: Public

Successful Response:

- Status Code: 200 OK
- JSON Example (List<ContentResponseDTO>):

```
[
  {
    "id": 1,
    "submoduleId": 1,
    "contentType": "TEXT",
    "title": "Understanding Event Pipelines",
    "richTextContent": "### Immutable Events\nEvent pipelines serve...",
    "mediaUrl": null,
    "contentOrder": 1,
    "durationMinutes": 10
  }
]
```

2. Create Content Block (Admin Authoring)

Endpoint: POST /admin/submodules/{submoduleId}/contents

Purpose: Adds a content asset to a lesson submodule.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Request Body (ContentRequestDTO):

```
{
  "title": "AWS Compute Engine Lab Instructions",
  "contentType": "DOCUMENT",
  "richTextContent": "Please read carefully...",
  "mediaUrl": "https://files.educorp.com/lab-manual.pdf",
  "contentOrder": 2,
  "durationMinutes": 15,
  "isActive": true
}
```

Successful Response:

- Status Code: 201 Created

Files

Handles local multi-part binary asset uploads and direct downloads.

1. Secure Binary File Upload

Endpoint: POST /files/upload

Purpose: Receives multipart files, saves them inside local server uploads storage using generated UUID file prefixes to prevent collisions, and returns a download link.

Authentication: JWT Required

Authorized Roles: Admin, Teacher, Student

Request Body:

- Form Parameter: file (Multipart Binary, Required)

Successful Response:

- Status Code: 200 OK

```
{
  "url": "http://localhost:8080/files/download/2a7a8d43-fc05-468b-bfa1_lab-report.pdf",
  "fileName": "lab-report.pdf"
}
```

Error Responses:

- 500 Internal Server Error: Failed to store file / Failed to upload file.

2. Download File

Endpoint: GET /files/download/{fileName}

Purpose: Serves locally stored files back to frontend players or browsers with mapped content-type headers. Strips the UUID prefix from downloaded filenames automatically.

Authentication: Public

Path Variables:

- fileName (String, Required): Mapped file string.

Successful Response:

- Status Code: 200 OK

Headers:

- Content-Type: application/octet-stream (or matched mime-type)
 - Content-Disposition: attachment; filename="lab-report.pdf"
-

Enrollments

Learner-facing course registrations.

1. Enroll Currently Authenticated Student in Course

Endpoint: POST /enrollments

Purpose: Self-registers the logged-in student into a course.

Authentication: JWT Required

Authorized Roles: Student

Request Body (EnrollmentRequestDTO):

```
{
  "courseId": 1
}
```

Successful Response:

- Status Code: 201 Created
- JSON Example (EnrollmentResponseDTO):

```
{
  "id": 105,
  "studentId": 2,
  "studentName": "Emma Watson",
  "courseId": 1,
  "courseTitle": "AWS Certified Solutions Architect",
  "status": "ENROLLED",
  "progressPercentage": 0,
  "enrollmentDate": "2026-07-13T18:47:40"
}
```

2. Get My Active Course Enrollments

Endpoint: GET /enrollments/me

Purpose: Lists all course registrations for the logged-in learner.

Authentication: JWT Required

Authorized Roles: Student

Successful Response:

- Status Code: 200 OK

3. Enroll in Batch (Direct Cohort Invite URL mapping)

Endpoint: POST /students/batches/{batchId}/enroll

Purpose: Facilitates instant cohort registration. Automatically mapping the custom batchId invite parameter (e.g. alphanumeric codes like compliance, cyber) to a backend Course ID, establishing a new active course enrollment.

Authentication: JWT Required

Authorized Roles: Student

Path Variables:

- batchId (String, Required): Batch code invitation token.

Successful Response:

- Status Code: 201 Created

Batches

Handles cohort mapping to organize student progress, map teachers, and restrict assignment viewing to specific student rosters.

1. Create Student Cohort (Batch)

Endpoint: POST /batches

Purpose: Registers a cohort linked to a specific course and teacher.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Request Body (BatchCreateRequestDTO):

```
{
  "name": "AWS Architecture - Cohort Alpha",
  "batchCode": "BATCH-AWS-OCT-2026",
  "courseId": 1,
  "startDate": "2026-10-01T08:00:00",
  "endDate": "2026-12-15T18:00:00",
  "isActive": true
}
```

Successful Response:

- Status Code: 201 Created
- JSON Example (BatchResponseDTO):

```
{
  "id": 1,
  "name": "AWS Architecture - Cohort Alpha",
  "batchCode": "BATCH-AWS-OCT-2026",
  "courseId": 1,
  "courseTitle": "AWS Certified Solutions Architect",
  "teacherId": 3,
  "teacherName": "Teacher User",
}
```

```
"startDate": "2026-10-01T08:00:00",
"endDate": "2026-12-15T18:00:00",
"isActive": true,
"studentCount": 0,
"createdAt": "2026-07-13T18:47:40",
"updatedAt": "2026-07-13T18:47:40"
}
```

2. Get My Cohorts (Contextual Feed)

Endpoint: GET /batches/my

Purpose: Returns batches assigned to the authenticated admin/teacher.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Successful Response:

- Status Code: 200 OK

3. Assign Student to Batch

Endpoint: POST /batches/{batchId}/students/{studentId}

Purpose: Enrolls a specific student into a cohort.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Path Variables:

- batchId (Long, Required): Database batch record ID.
- studentId (Long, Required): Database student user ID.

Successful Response:

- Status Code: 201 Created
- JSON Example (BatchStudentResponseDTO):

```
{
  "id": 12,
  "batchId": 1,
  "studentId": 2,
  "studentName": "Emma Watson",
  "assignedAt": "2026-07-13T18:47:40"
}
```

4. Remove Student from Batch

Endpoint: DELETE /batches/{batchId}/students/{studentId}

Purpose: Unassigns a student from a cohort.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Successful Response:

- Status Code: 204 No Content
-

Assignments

Manages student grading checkpoints. Facilitates extensive customization options (shuffle, negative marking, certificate bindings).

1. Create Assignment

Endpoint: POST /assignments

Purpose: Creates a new evaluation template under a course.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Request Body (AssignmentCreateRequestDTO):

```
{
  "title": "AWS Cloud Security Checkpoint",
  "description": "Assess IAM configurations and AWS KMS policies.",
  "instructions": "Answer all multiple-choice questions. Time limit is 30 minutes.",
  "courseId": 1,
  "teacherId": 3,
  "status": "DRAFT",
  "availableFrom": "2026-07-15T00:00:00",
  "dueDate": "2026-08-01T23:59:59",
  "maxMarks": 100,
  "passingMarks": 50,
  "timeLimitMinutes": 30,
  "allowLateSubmission": false,
  "shuffleQuestions": true,
  "showResultImmediately": true,
  "maxAttempts": 1,
  "autoSubmit": true,
  "shuffleOptions": true,
  "negativeMarking": true,
  "showDetailedAnswers": false,
  "enableCertificates": true,
  "batchIds": [1, 2]
}
```

Successful Response:

- Status Code: 201 Created
- JSON Example (AssignmentResponseDTO):

```
{
  "id": 1,
  "title": "AWS Cloud Security Checkpoint",
  "description": "Assess IAM configurations.",
  "instructions": "Answer all questions.",
  "courseId": 1,
  "courseTitle": "AWS Certified Solutions Architect",
  "teacherId": 3,
  "teacherName": "Teacher User",
  "status": "DRAFT",
  "availableFrom": "2026-07-15T00:00:00",
  "dueDate": "2026-08-01T23:59:59",
  "maxMarks": 100,
  "passingMarks": 50,
}
```

```

    "timeLimitMinutes": 30,
    "allowLateSubmission": false,
    "shuffleQuestions": true,
    "showResultImmediately": true,
    "isActive": true,
    "maxAttempts": 1,
    "autoSubmit": true,
    "shuffleOptions": true,
    "negativeMarking": true,
    "showDetailedAnswers": false,
    "enableCertificates": true,
    "assignedBatchIds": [1, 2],
    "assignedBatchNames": ["AWS Cohort 1", "AWS Cohort 2"],
    "questionCount": 0,
    "createdAt": "2026-07-13T18:47:40",
    "updatedAt": "2026-07-13T18:47:40"
  }

```

2. Get My Assignments (Student Feed)

Endpoint: GET /assignments/student/me

Purpose: Lists active published assignments for courses the student is actively enrolled in.

Authentication: JWT Required

Authorized Roles: Student

Successful Response:

- Status Code: 200 OK

3. Bind Batches to Assignment (Cohort Targeting)

Endpoint: POST /assignments/{id}/batches

Purpose: Targets an assignment specifically to certain student cohorts.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Path Variables:

- id (Long, Required): Database assignment record ID.

Query Parameters:

- batchId (Long, Optional): Single batch database ID to bind.

Request Body (Optional):

- List<Long>: JSON array of batch IDs to bind in a batch action (e.g. [1, 2, 3]).

Successful Response:

- Status Code: 200 OK

Questions

Interactive quiz builder supporting Multiple Choice (MCQ), Multiple Correct, Essay (Paragraph), and File Upload evaluations.

1. Create Question

Endpoint: POST /questions

Purpose: Appends a question with options (answers) to an assignment template.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Request Body (QuestionCreateRequestDTO):

```
{
  "assignmentId": 1,
  "title": "Which AWS storage class is optimal for raw archival files accessed once a year?",
  "description": "Select the most cost-efficient option.",
  "questionType": "MCQ",
  "marks": 5,
  "displayOrder": 1,
  "required": true,
  "negativeMarks": 1,
  "options": [
    {
      "optionText": "S3 Standard",
      "isCorrect": false,
      "displayOrder": 1
    },
    {
      "optionText": "S3 Glacier Deep Archive",
      "isCorrect": true,
      "displayOrder": 2
    }
  ]
}
```

Successful Response:

- Status Code: 201 Created
- JSON Example (QuestionResponseDTO):

```
{
  "id": 1,
  "assignmentId": 1,
  "title": "Which AWS storage class is optimal for raw archival files accessed once a year?",
  "description": "Select the most cost-efficient option.",
  "questionType": "MCQ",
  "marks": 5,
  "displayOrder": 1,
  "required": true,
  "negativeMarks": 1,
  "options": [
    {
      "id": 1,
      "optionText": "S3 Standard",
      "isCorrect": false,
      "displayOrder": 1
    },
    {
      "id": 2,
```

```
    "optionText": "S3 Glacier Deep Archive",
    "isCorrect": true,
    "displayOrder": 2
  }
]
```

2. Get Questions by Assignment

Endpoint: GET /questions/assignment/{assignmentId}

Purpose: Fetches the questions list. When queried by a student, correct answer flags (isCorrect) are automatically omitted from options to prevent cheating.

Authentication: JWT Required

Authorized Roles: Admin, Teacher, Student

Successful Response:

- Status Code: 200 OK

Submissions & Auto-Grading

Encompasses student exam sessions, automated MCQ correction engines, and teacher grading dashboards.

1. Initialize (Start) Exam Session

Endpoint: POST /submissions/start

Purpose: Spawns a new active exam tracking token containing a synchronized startedAt timestamp. Validates user attempt bounds.

Authentication: JWT Required

Authorized Roles: Student

Request Body (SubmissionCreateRequestDTO):

```
{
  "assignmentId": 1,
  "studentId": 2
}
```

Successful Response:

- Status Code: 201 Created
- JSON Example (SubmissionResponseDTO):

```
{
  "id": 15,
  "assignmentId": 1,
  "assignmentTitle": "AWS Cloud Security Checkpoint",
  "studentId": 2,
  "studentName": "Emma Watson",
  "status": "IN_PROGRESS",
  "startedAt": "2026-07-13T18:47:40",
  "totalMarks": 100,
  "obtainedMarks": 0,
}
```

```
"answers": []
}
```

2. Submit Active Session (MCQ Auto-Grading Core)

Endpoint: POST /submissions/submit

Purpose: Locks answers, calculates correct/incorrect options immediately, registers negative marking bounds, stamps a submittedAt marker, and sets final scores if MCQ-only.

Authentication: JWT Required

Authorized Roles: Student, Admin, Teacher

Query Parameters:

- id or submissionId (Long, Required): Database submission record ID.

Request Body (SubmissionUpdateRequestDTO):

```
{
  "status": "SUBMITTED",
  "timeTakenMinutes": 22,
  "answers": [
    {
      "questionId": 1,
      "selectedOptionId": 2,
      "answerText": null
    }
  ]
}
```

Successful Response:

- Status Code: 200 OK
- JSON Example (SubmissionResponseDTO):

```
{
  "id": 15,
  "assignmentId": 1,
  "status": "COMPLETED",
  "startedAt": "2026-07-13T18:47:40",
  "submittedAt": "2026-07-13T19:09:40",
  "totalMarks": 100,
  "obtainedMarks": 85,
  "percentage": 85.00,
  "timeTakenMinutes": 22,
  "isLateSubmission": false,
  "answers": [
    {
      "id": 45,
      "questionId": 1,
      "selectedOptionId": 2,
      "isCorrect": true,
      "obtainedMarks": 5
    }
  ]
}
```

3. Save Draft (Draft Persistence)

Endpoint: PUT /submissions/save

Purpose: Temporarily saves active answers mid-exam without triggering final grading/attempt validation.

Authentication: JWT Required

Authorized Roles: Student

Query Parameters:

- `id` or `submissionId` (Long, Required): Submission database ID.

Request Body: Same as `SubmissionUpdateRequestDTO`.

Successful Response:

- Status Code: 200 OK

4. Post manual grades (Teacher Evaluation)

Endpoint: POST `/submissions/{id}/grade`

Purpose: Evaluates essay questions, overrides marks, and provides structured feedback comments.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Request Body (`SubmissionUpdateRequestDTO`):

```
{
  "status": "GRADED",
  "obtainedMarks": 95,
  "answers": [
    {
      "questionId": 2,
      "obtainedMarks": 10,
      "teacherFeedback": "Excellent depth of coverage on micro-segmentation
principles."
    }
  ]
}
```

Successful Response:

- Status Code: 200 OK

Certificates

Certificates are a built-in business automation rule driven by active assignment parameters:

- If the field `enableCertificates` is set to `true` on an active assignment, the system activates automated certification evaluation.
- When a student's final submission is graded (or auto-graded), the system compares their score (`obtainedMarks`) against the course or assignment `passingMarks`.
- If the student meets or exceeds the required passing marks threshold, they are flagged as Eligible for Certificate.
- Eligibility is marked with a "Yes" or "No" status, which is included directly in programmatic reports, spreadsheets, and the spreadsheet-export module.

Export

Enables admins and teachers to download formatted Excel spreadsheet reports containing grading and class roster databases.

1. Export Assignment Results to Excel Spreadsheet

Endpoint: GET /export/assignment/{assignmentId}

Purpose: Compiles a custom Excel spreadsheet containing student grades, completion times, dates, percentages, and certificate eligibility metrics.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Path Variables:

- assignmentId (Long, Required): Target assignment template ID.

Successful Response:

- Status Code: 200 OK

Headers:

- Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
- Content-Disposition: attachment; filename=Assignment_Results_Export_1.xlsx

2. Export Cohort (Batch) Student Directory to Excel

Endpoint: GET /export/batch/{batchId}

Purpose: Compiles a master directory of students actively mapped to a cohort.

Authentication: JWT Required

Authorized Roles: Admin, Teacher

Successful Response:

- Status Code: 200 OK

Headers:

- Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
- Content-Disposition: attachment; filename=Batch_AWS_Solutions_Architect_Cohort_1_Students.xlsx

Quality Assurance Checklist

The following verifications have been conducted on the documentation:

- **Controller Verification:** Verified that every mapped controller aligns exactly with active Java classes in `com.geeknito.geeknito_backend`.
- **Path Accuracy:** Double-checked multiple request mappings (`/auth`, `/api/categories`, `/catalog/courses`, `/enrollments`, `/batches`, `/export`, etc.) for absolute parity.
- **DTO Properties:** Explicitly checked fields inside Java source files (`AssignmentCreateRequestDTO.java`, `SubmissionResponseDTO.java`, etc.) to prevent mismatches in field names, annotations, or JSON property tags.
- **No Phantom Routes:** Excluded any unassigned routes. Documented only actual REST services existing in the codebase.
- **Authentication Matrix:** Cross-checked controller filters, security principal validation helper triggers (`getAuthenticatedUser`), and user context roles with standard security structures.